COA PROJECT 2026

Implementation of Traffic Signal Controller

- Visualizing Memory Mapped I/O and its Hardware Implementation -

INTRODUCTION & OBJECTIVES

The core challenge in studying Computer Organization and Architecture is bridging the gap between static assembly code and the dynamic behavior of the processor. Concepts like the 5-stage pipeline, data hazards, and memory addressing often remain abstract. The objective of this project is to demystify these operations by designing a Traffic Signal Controller using the **Ripes** simulator. Unlike standard coding, this project requires direct manipulation of the processor's memory to control peripherals. By simulating a complex Dual-Post intersection, we aim to visualize exactly how a RISC-V processor fetches, decodes, and executes I/O instructions in real-time.

MEMORY MAPPED I/O & PERIPHERAL INEGRATION

The interface between the assembly code and the traffic peripherals was implemented through **Memory-Mapped I/O (MMIO)**. In this architecture, the processor treats the traffic light hardware as a specific memory address, specifically **0xF0000000**. The simulation uses a **1x2 LED Matrix** to represent the intersection. By executing a sw (Store Word) instruction, the processor sends a bitmask—such as 0x01 for Green or 0x04 for Red—directly to this address. This method demonstrates the "Digital Twin" concept, where the virtual environment accurately reflects how a physical CPU would manipulate electrical signals to control real-world infrastructure

IMPLEMENTATION & ASSEMBLY LOGIC

The control logic is implemented entirely in RISC-V Assembly language. We designed a state machine that manages a Dual Traffic-Post Intersection. The program utilizes a specific memory offset strategy—using base-plus-offset addressing—to target two distinct clusters of LEDs on the matrix simultaneously. The code is structured into four distinct phases, using a looping mechanism to cycle through Green, Yellow, and Red states. Crucially, we implemented software-based delay loops using subtraction instructions (addi) to synchronize the processor's high-speed execution with human-perceptible timing, ensuring the traffic lights blink at realistic intervals.

ARCHITECTURAL VISUALIZATION: The 5-Stage Pipeline

- **IF (Instruction Fetch):** Traffic logic retrieved from instruction memory via Program Counter.
- **ID (Instruction Decode):** Translating sw and addi instructions while reading register values.
- **EX (Execute):** ALU calculating timing offsets and decrementing delay counters.
- **MEM (Memory Access):** The critical stage where the MMIO signal is sent to address 0xF0000000.
- **WB (Write Back):** Updating registers for the next state transition.

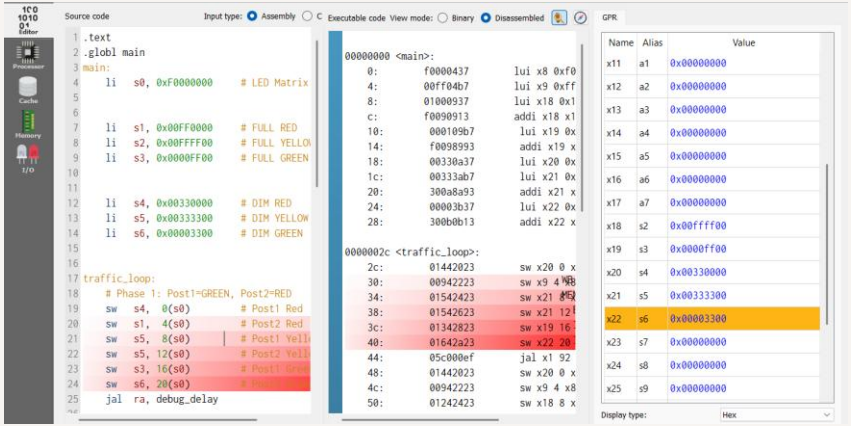
PIPELINE EXECUTION & ARCHITECTURAL SIMULATION

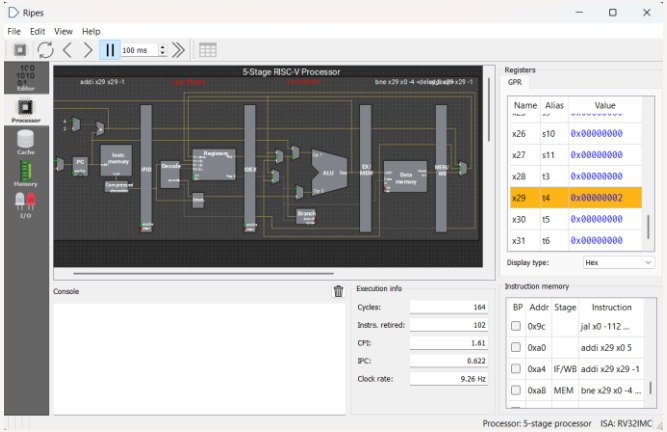
The core of the simulation was conducted using the **Ripes RISC-V 5-stage pipeline model**, which allowed for a transparent view of the processor's internal datapath. During execution, the traffic control logic is not processed as a single block but is broken down into five distinct stages: **Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB)**. By slowing the simulation clock frequency, we were able to visualize instruction-level parallelism, where a store instruction (sw) updating the traffic light status would occupy the Memory stage while the next delay-loop instruction was already being processed in the Execute stage. This architectural analysis confirms that the traffic system is managed by a high-throughput processor capable of handling multiple instructions simultaneously without data hazards.

SIMULATION & PHYSICAL HARDWARE

To validate the simulation in a real-world context, a physical hardware mirror was developed using an **Arduino-based microcontroller**. The system functioned as a "Digital Twin," where the state changes observed in the Ripes pipeline were transmitted via serial communication to the physical LEDs. Because the RISC-V processor operates at significantly higher frequencies than human-readable traffic cycles, **software-based delay loops** were implemented using subtraction and branch instructions. These loops effectively synchronized the high-speed execution of the 5-stage pipeline with the physical requirements of a traffic system, ensuring that each signal phase remained stable for the designated duration before the next instruction was retired in the pipeline.

SIMULATION RESULTS





TIMING CALIBRATION & PHYSICAL HARDWARE MIRRORING

To ensure the simulation results were applicable to real-world scenarios, the project integrated **physical hardware mirroring** via an Arduino microcontroller. Because the RISC-V pipeline operates at speeds far exceeding human perception, **software-based delay loops** were designed using subtraction and branch logic. These loops effectively "pause" the state machine by forcing the processor to execute a set number of cycles before transitioning the light phase. This synchronization ensures that the timing observed in the Ripes simulator perfectly matches the physical LED transitions on the hardware. This dual-layer approach—combining virtual pipeline analysis with physical verification—validates the system's reliability as a functional traffic signal controller.

CONCLUSION

The implementation of the Traffic Signal Controller demonstrates a seamless integration of RISC-V assembly and a **5-stage pipelined architecture**, providing a transparent view of the instruction lifecycle through the **Fetch, Decode, Execute, Memory, and Write-back** stages. By utilizing **Memory-Mapped I/O (MMIO)** at address **0xF0000000**, the software successfully controls a simulated 1x2 LED Matrix, where each sw (store word) instruction acts as a direct link between the processor's datapath and hardware state changes. To bridge the gap between high-speed simulation and human perception, software-based delay loops were calibrated to slow execution to real-time intervals, effectively creating a **"Digital Twin"** model where the virtual Ripes environment and the physical Arduino-based hardware remain perfectly synchronized.

THANK YOU!

- TEAM **MEMBERS -** Kashinath R Krishna - Kannan Ramanan - Jeevan Krishnan - Arjun Prasad - Devananda S - Neehara Anil KK